

Manual de usuario

SOLVER BIO-INSPIRADOS

José Lara Arce

Pontificia Universidad Católica de Valparaíso, Chile

`jose.lara.a01@mail.pucv.cl`

July 1, 2026

Índice

1. Dependencias	2
1.1. Crear y Activar el entorno virtual con venv	2
1.2. Configurar Visual Studio Code	3
1.3. Instalar dependencias	3
2. Introducción al solver	4
3. Estructura interna del solver	6
3.1. Archivos y Scripts principales	6
4. Ejemplo de uso y explicación	7
4.1. Optimización de funciones matemáticas (Benchmark) y Problemas Binarios	7
4.1.1. Paso 1: Configuración del archivo <code>experiments_config.json</code>	7
4.1.2. Paso 2: Poblar la base de datos de experimentos	9
4.1.3. Paso 3: Ejecutar el solucionador	9
5. Resultados y Base de Datos	10
5.1. Estructura de la Base de Datos	10
5.2. Estructura de Directorios de Salida	12
6. Agregar una nueva metaheurística	12
6.1. Paso 1: Implementación de la lógica del algoritmo	12
6.2. Paso 2: Registro en el sistema	14
6.3. Paso 3: Habilitar en experimentos	15
6.4. Paso 4: Consideraciones adicionales (opcional)	15
7. Análisis de resultados	15
7.1. Estructura del script de análisis	15
7.2. Configuración del análisis	16
7.3. Ejecución del análisis	17
7.4. Salidas generadas	17

1. Dependencias

Para el correcto funcionamiento del solver, es necesario contar con los siguientes requisitos:

- **Versión de Python:** Python 3.8 o superior (se recomienda Python 3.11+ por rendimiento). El framework es totalmente compatible con cualquier versión moderna de Python (incluyendo Python 3.12 y superiores) sin restricciones de compatibilidad.
- **IDEs recomendados:** Visual Studio Code, PyCharm.
- **Gestión de dependencias:** Se utiliza el módulo `venv`, incluido en la biblioteca estándar de Python (desde la versión 3.3), para crear entornos virtuales de forma aislada.

Nota: La creación de un entorno virtual es opcional. Si no desea aislar las dependencias del proyecto y prefiere instalar los paquetes de Python directamente en el entorno global del sistema, puede omitir los pasos de creación y activación del entorno virtual y proceder directamente a la sección 1.3. Sin embargo, se recomienda encarecidamente el uso de un entorno virtual para evitar conflictos entre versiones de librerías en diferentes proyectos.

1.1. Crear y Activar el entorno virtual con venv

El módulo `venv` permite crear entornos virtuales aislados para gestionar paquetes sin afectar otras configuraciones del sistema. Para ello:

1. Navegue hasta el directorio de su proyecto en la terminal o línea de comandos. 2. Cree un entorno virtual llamado `env` (puede elegir otro nombre si lo prefiere) con el siguiente comando:

```
python -m venv env
```

3. Una vez creado, active el entorno virtual. El comando varía según su sistema operativo y terminal:

- **Windows (cmd.exe):**

```
env\Scripts\activate.bat
```

- **Windows (PowerShell):**

```
.\env\Scripts\Activate.ps1
```

(Nota: Es posible que necesite ajustar la política de ejecución de scripts en PowerShell con `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope Process`).

■ **Unix o MacOS (bash/zsh):**

```
source env/bin/activate
```

Al activarse correctamente, la línea de comandos debería mostrar el nombre del entorno virtual entre paréntesis al inicio, similar a:

```
(env) C:\Users\Usuario\Proyecto>
```

o

```
(env) usuario@computador:~/Proyecto$
```

1.2. Configurar Visual Studio Code

Para asegurarse de que Visual Studio Code use el intérprete de Python correcto dentro del entorno virtual:

1. Abra la **Command Palette** (Paleta de Comandos) con:

```
Ctrl + Shift + P
```

(o `Cmd + Shift + P` en macOS). 2. Escriba `>Python: Select Interpreter` (o `>Python: Seleccionar Intérprete`) y presione Enter. 3. Seleccione la opción que apunta al ejecutable de Python dentro de su entorno virtual. Usualmente contendrá la ruta `./env/bin/python` (Unix/macOS) o `.\Scripts\python.exe` (Windows). VS Code a menudo detecta y sugiere automáticamente los entornos `venv`.

1.3. Instalar dependencias

Para instalar las dependencias del proyecto, asegúrese de que el entorno virtual `(env)` esté activado (verá `(env)` al inicio de la línea de comandos) y ejecute en la consola:

```
pip install -r requirements.txt
```

2. Introducción al solver

El solver que presentamos en este manual está diseñado para resolver problemas de optimización utilizando técnicas de metaheurísticas. Estas técnicas son métodos de búsqueda de soluciones aproximadas para problemas complejos donde los algoritmos exactos resultan computacionalmente inviables. En particular, el framework implementa las siguientes 37 metaheurísticas hasta la fecha:

- **ALA:** Artificial Lemming Algorithm
- **AOA:** Arithmetic Optimization Algorithm
- **APO:** Artificial Protozoa Optimization
- **CCMGO:** Criss-Cross Mountain Gazelle Optimizer
- **CLO:** Cape Lynx Optimizer
- **DHOA:** Dhole Optimization Algorithm
- **DLO:** Draco Lizard Optimizer
- **DOA:** Dream Optimization Algorithm
- **DRA:** Divine Religion Algorithm
- **EBWOA:** Enhanced Beluga Whale Optimization Algorithm
- **EHO:** Elk Herd Optimizer
- **EOO:** Eurasian Oystercatcher Optimizer
- **FLO:** Frilled Lizard Optimization
- **FOX:** Fox-inspired Optimization Algorithm
- **GA:** Genetic Algorithm
- **GOA:** Gannet Optimization Algorithm
- **GOAT:** Goat Optimization Algorithm
- **GWO:** Grey Wolf Optimizer
- **HBA:** Honey Badger Algorithm
- **HLOA:** Horned Lizard Optimization Algorithm (con variantes BEN y SCP)
- **LOA:** Lyrebird Optimization Algorithm

- **NO:** Narwhal Optimization
- **PGA:** Phototropic Growth Algorithm
- **POA:** Pufferfish Optimization Algorithm
- **PSA:** Pendulum Search Algorithm
- **PSO:** Particle Swarm Optimization
- **QSO:** Quokka Optimization Algorithm
- **RSA:** Reptile Search Algorithm
- **SBOA:** Secretary Bird Optimization Algorithm
- **SCA:** Sine Cosine Algorithm
- **SHO:** Sea-Horse Optimizer
- **SSO:** Salmon Salar Optimization
- **TDO:** Tasmanian Devil Optimization
- **TJO:** Traffic Jam Optimization
- **WO:** Walrus Optimizer
- **WOA:** Whale Optimization Algorithm
- **WOM:** Wombat Optimization Algorithm

El solver es altamente flexible y modular, permitiendo optimizar diferentes funciones objetivo y problemas. Actualmente, cuenta con soporte integrado para los siguientes tipos de problemas:

- **Benchmark (BEN):** Funciones matemáticas continuas, incluyendo el benchmark estándar CEC2017.
- **Set Covering Problem (SCP):** Problema de cobertura de conjuntos clásico (discreto/binario).
- **Unicost Set Covering Problem (USCP):** Problema de cobertura de conjuntos con costos unitarios (discreto/binario).
- **Knapsack Problem (KP):** Problema de la mochila (discreto/binario), el cual ha sido incorporado recientemente al framework.

Las configuraciones específicas de los experimentos se definen de manera centralizada a través de archivos de configuración JSON.

3. Estructura interna del solver

El solver está organizado de manera limpia y modular bajo el directorio `src/` y otros directorios auxiliares con el objetivo de facilitar su expansión. A continuación, se describen los componentes clave:

- **src/bd**: Módulo encargado de la gestión de la base de datos SQLite (`sqlite.py`) y la definición del esquema de experimentos.
- **src/chaotic_maps**: Implementa mapas caóticos optimizados con Numba JIT para perturbar las soluciones en optimización.
- **src/discretization**: Contiene las funciones de transferencia y los métodos de binarización de soluciones.
- **src/diversity**: Contiene funciones para evaluar la diversidad de la población y el balance entre exploración y explotación.
- **src/metaheuristics**: Contiene los archivos Python individuales de cada algoritmo bajo `Codes/`, centralizados mediante `imports.py`.
- **src/problem**: Define las estructuras y métodos de evaluación para los problemas de Benchmark (CEC2017), SCP, USCP y KP.
- **src/solver**: Contiene el `universal_solver.py`, adaptadores de firma y el despachador de experimentos.
- **src/util**: Utilidades globales de configuración JSON (`src/util/json/`), logging y utilidades gráficas.
- **data**: Directorio de datos que almacena la base de datos local (`data/database/resultados`) y las instancias de entrada de los problemas (`data/instances/`).
- **outputs**: Almacena logs de ejecución, resúmenes CSV y gráficos generados de forma automática (ignorados por Git).
- **tests**: Suite de pruebas unitarias implementadas con `pytest`.

3.1. Archivos y Scripts principales

El framework divide los puntos de entrada para la ejecución de experimentos de aquellos dedicados a la administración y análisis:

- **main.py**: El ejecutable principal que coordina el flujo secuencial del solucionador.

- **main_joblib.py**: Script ejecutable para correr experimentos en paralelo en la máquina local usando el framework Joblib, optimizando el uso de núcleos de CPU de forma concurrente.
- **levantarCMD.py**: Permite la creación y ejecución de múltiples instancias del solver en paralelo levantando consolas CMD individuales (ideal para Windows).
- **scripts/analisis.py**: Script orquestador del análisis de resultados. Carga el módulo de procesamiento modular de `analisis_pkg` para generar tablas comparativas, boxplots y gráficos de convergencia a partir de los datos en SQLite.
- **scripts/crearBD.py**: Inicializa el esquema de la base de datos SQLite si esta no existe.
- **scripts/poblarDB.py**: Lee la configuración del archivo `experiments_config.json` e inserta los experimentos a ejecutar en la base de datos local.
- **scripts/reiniciarDB.py**: Reinicia la base de datos eliminando los experimentos e insertando la configuración por defecto.
- **scripts/limpiarEntorno.py**: Elimina los logs y resultados generados en ejecuciones anteriores en el directorio `outputs/`.
- **scripts/reiniciarSolver.py**: Restablece el solver por completo a su estado inicial, ejecutando tanto `limpiarEntorno.py` como `reiniciarDB.py`.

4. Ejemplo de uso y explicación

A continuación, se presenta un ejemplo detallado de cómo utilizar el solver para la optimización de funciones matemáticas o problemas binarios utilizando el archivo de configuración centralizado.

4.1. Optimización de funciones matemáticas (Benchmark) y Problemas Binarios

Para comenzar con la optimización, lo primero que deben hacer es configurar las instancias, algoritmos y parámetros en el archivo de configuración.

4.1.1. Paso 1: Configuración del archivo `experiments_config.json`

Diríjase al directorio `src/util/json/` y abran el archivo `experiments_config.json`. Dentro de este archivo encontrarán la estructura que controla la ejecución. A continuación, se detallan las secciones clave:

■ Selección del tipo de problema

Para habilitar o deshabilitar problemas específicos, se utilizan llaves booleanas. Por ejemplo, para ejecutar optimización en Benchmark (BEN) y desactivar SCP, USCP y KP:

```
"ben": true,
"scp": false,
"uscp": false,
"kp": false,
```

■ Selección de metaheurísticas

En la lista "mhs" se declaran las siglas de los algoritmos a ejecutar (por ejemplo, SBOA, PSO, GWO):

```
"mhs": ["SBOA"],
```

■ Configuración de acciones de discretización

La lista "DS_actions" define las binarizaciones para problemas discretos (como SCP, USCP o KP). Esto no influye en las funciones continuas (BEN), pero es crucial para los problemas discretos:

```
"DS_actions": ["V3-ELIT"],
```

■ Configuración de Mapas Caóticos (Novedad)

El framework permite ejecutar experimentos usando mapas caóticos en las ecuaciones de perturbación. Se puede activar globalmente y definir modos como "comparacion" (ejecuta tanto el método estándar como el caótico para comparar resultados):

```
"usar_mapas_caoticos": true,
"mapas_caoticos": {
  "modo": "comparacion",
  "mapas_activos": ["LOG", "SINE"],
  "SCP": {
    "usar_caoticos": true,
    "modo": "comparacion",
    "mapas": ["LOG"]
  }
}
```

■ Criterios de Parada y Parámetros de Ejecución

En la sección ".experimentos" se especifica el criterio de parada global (modo_terminacion: "iter" para iteraciones, "fe" para evaluaciones de función, o "both" para ambas), el límite global de evaluaciones ("max_fe") y los parámetros específicos para cada dominio:

```

"experimentos": {
  "modo_terminacion": "iter",
  "max_fe": 50000,
  "BEN": {
    "iteraciones": 500,
    "poblacion": 50,
    "num_experimentos": 1
  },
  "SCP": {
    "iteraciones": 100,
    "poblacion": 10,
    "num_experimentos": 31
  }
}

```

■ Selección de instancias a optimizar

En la sección "instancias" se indica la lista exacta de instancias o funciones por cada tipo de problema. Por ejemplo, para Benchmark y SCP:

```

"instancias": {
  "BEN": ["F1CEC2017", "F2CEC2017", "F3CEC2017"],
  "SCP": ["41"],
  "USCP": ["u41"],
  "KP": []
}

```

4.1.2. Paso 2: Poblar la base de datos de experimentos

Una vez configurado el archivo `experiments_config.json`, es necesario inyectar los experimentos parametrizados en la base de datos de SQLite antes de poder ejecutarlos. Para ello, ejecuten en la consola:

```

python scripts/crearBD.py
python scripts/poblarDB.py

```

El script `poblarDB.py` se encargará de crear los registros de cada corrida para cada metaheurística e instancia seleccionada en la base de datos.

4.1.3. Paso 3: Ejecutar el solucionador

Una vez poblada la base de datos, tienen tres alternativas para ejecutar el solver según la conveniencia de su máquina:

1. Ejecución secuencial:

```
python main.py
```

Ejecuta un experimento a la vez secuencialmente mostrando el progreso por consola.

2. Ejecución paralela local (Joblib):

```
python main_joblib.py --n-jobs 4
```

Utiliza el framework de paralelismo Joblib para competir de forma segura por los experimentos en la base de datos, acelerando drásticamente el proceso.

3. Ejecución distribuida en consolas (Windows):

```
python levantarCMD.py
```

Este script abre múltiples terminales CMD ejecutando `main.py` de manera concurrente para paralelizar el consumo de la base de datos.

5. Resultados y Base de Datos

El solver utiliza un esquema unificado de base de datos relacional basado en **SQLite** para coordinar la ejecución distribuida de los experimentos y almacenar de forma persistente los resultados detallados de cada corrida.

La base de datos local se genera de manera automática al inicializar el solver y se almacena en el directorio `data/database/resultados.db`.

5.1. Estructura de la Base de Datos

El diseño relacional consta de cuatro tablas principales que se describen a continuación:

1. **Tabla `instancias`:** Almacena las especificaciones de las funciones matemáticas o instancias de problemas que el solver optimizará.
 - `id_instancia` (INTEGER, PK): Identificador único de la instancia.
 - `tipo_problema` (TEXT): Dominio del problema (ej. "BEN", "SCP", "USCP", "KP").

- `nombre` (TEXT): Nombre específico (ej. "F1CEC2017", "41", "ü41").
- `optimo` (REAL): Valor óptimo global conocido (utilizado para calcular brechas de calidad o GAPS).
- `param` (TEXT): Parámetros de frontera de búsqueda (como límites `lb` y `ub`).

2. **Tabla `experimentos`:** Actúa como cola de trabajos para los workers paralelos. Representa cada combinación única de algoritmo, problema, binarización y parámetros.

- `id_experimento` (INTEGER, PK): Identificador único del experimento.
- `experimento` (TEXT): Nombre representativo de la configuración.
- `MH` (TEXT): Sigla de la metaheurística a ejecutar (ej. "SBOA", "PSO").
- `binarizacion` (TEXT): Nombre de la función de transferencia y binarización (ej. "V3-ELIT"), o nombre del mapa caótico acoplado (ej. "V3-ELIT_LOG").
- `estado` (TEXT): Estado actual de ejecución ("pendiente", "ejecutando", "finalizado"). Esto evita la concurrencia entre workers paralelos.
- `fk_id_instancia` (INTEGER, FK): Relación con la tabla `instancias`.
- `ts_inicio / ts_fin` (TEXT): Registros de marca de tiempo (timestamp) de inicio y fin.

3. **Tabla `resultados`:** Almacena el resumen del rendimiento final obtenido tras finalizar las iteraciones del experimento.

- `id_resultado` (INTEGER, PK): Identificador único del resultado.
- `fitness` (REAL): El mejor valor de fitness final encontrado.
- `tiempoEjecucion` (REAL): El tiempo total de procesamiento en segundos.
- `solucion` (TEXT): Vector de la mejor solución codificado como cadena de texto.
- `fk_id_experimento` (INTEGER, FK): Relación con la tabla `experimentos`.

4. **Tabla `iteraciones`:** Almacena la evolución detallada paso a paso de la corrida para realizar análisis de convergencia y exploración/explotación.

- `id_archivo` (INTEGER, PK): Identificador único de la iteración.
- `nombre` (TEXT): Nombre del archivo lógico CSV asociado.
- `archivo` (BLOB): Contenido comprimido en formato CSV con los registros históricos de cada iteración (incluye columnas de `iter`, `best_fitness`, `time`, `XPL`, `XPT`, `ENT`, etc.).
- `fk_id_experimento` (INTEGER, FK): Relación con la tabla `experimentos`.

5.2. Estructura de Directorios de Salida

Además del almacenamiento persistente en SQLite, la ejecución genera archivos de salida organizados en el directorio `outputs/`:

- **`outputs/logs/`**: Logs históricos detallados de la consola y errores en tiempo de ejecución.
- **`outputs/results/`**:
 - `transitorio/`: Almacenamiento temporal de archivos CSV durante el procesamiento distribuido en Joblib.
 - `resumen/`: Tablas estadísticas resumidas generadas por el script de análisis.
 - `graficos/`: Curvas de evolución de fitness, porcentajes de exploración/explotación (`%XPL` e `%XPT`) y tiempo por iteración.
 - `boxplot/` / `violinplot/`: Gráficos de distribución estadística comparativa del rendimiento de las metaheurísticas para cada instancia analizada.

6. Agregar una nueva metaheurística

El diseño modular del solver facilita la incorporación de nuevas metaheurísticas bio-inspiradas o de otro tipo. Para añadir un nuevo algoritmo, sigue estos pasos:

6.1. Paso 1: Implementación de la lógica del algoritmo

1. Crear el archivo Python:

Navega hasta el directorio `src/metaheuristics/Codes/`. Crea un nuevo archivo Python para tu metaheurística (ej. `NAE.py`).

2. Definir la función de iteración:

Dentro de tu nuevo archivo, define la función principal que ejecutará una iteración del algoritmo (ej. `iterarNAE(...)`).

3. Establecer la firma estándar de la función:

Es **crucial** que tu función utilice los nombres de parámetros estandarizados que espera la función `iterate_population`. La firma general podría ser:

```

1 import numpy as np
2 # otras importaciones...
3
4 def iterarNAE(maxIter, iter, dim, population, fitness,
5             best, fo=None, lb=None, ub=None, lb0=None, ub0=None,
6             vel=None, pBest=None, objective_type='MIN')
7
8     """
9     Realiza una iteracion del Nuevo Algoritmo de Ejemplo
10    (NAE) .
11
12    Args:
13        maxIter (int): Maximo de iteraciones.
14        iter (int): Iteracion actual.
15        dim (int): Dimension del problema.
16        population (np.ndarray): Poblacion actual.
17        fitness (np.ndarray): Fitness de la poblacion
18        actual.
19        best (np.ndarray): Mejor solucion global hasta
20        ahora.
21        fo (callable, optional): Funcion objetivo.
22        Defaults to None.
23        lb0 (float, optional): Limite inferior escalar.
24        Defaults to None.
25        ub0 (float, optional): Limite inferior escalar.
26        Defaults to None.
27        ...
28
29    Puede tener mas argumentos segun la metaheuristica a
30    implementar.
31
32    Returns:
33        np.ndarray or tuple: Nueva poblacion, o tupla (
34        pob, vel/mejoras).
35    """
36    N = population.shape[0]
37    new_population = population.copy()
38
39    # -----
40    #     LOGICA DE TU METAHEURISTICA NAE
41    # -----
42
43    # Ejemplo placeholder:
44    # for i in range(N):
45    #     r = np.random.rand(dim)
46    #     new_population[i] = population[i] + r * (best
47    # - population[i])

```

```

37
38     # --- Valor de Retorno ---
39     return new_population
40     # 0: return new_population, new_vel
41     # 0: return new_population, posibles_mejoras
42

```

Listing 1: Ejemplo firma estándar (NAE) Mejorada

Importante:

- No todos los parámetros son necesarios para cada algoritmo.
- Usa los nombres estándar exactos (`iter`, `best`, `fo`, etc.).
- Devuelve la nueva población (y opcionalmente velocidad o mejoras).
- Para SCP/USCP, la función opera en continuo; la binarización/repación es posterior.

6.2. Paso 2: Registro en el sistema

Registra la nueva función:

1. Editar `src/metaheuristics/imports.py`:**2. Añadir importación:**

```

1 from .Codes.NAE import iterarNAE # Ajusta NAE
2

```

3. Añadir a `metaheuristics`:

```

1 metaheuristics = {
2     # ...
3     "NAE": iterarNAE, # Nueva entrada
4     # ...
5 }
6

```

4. Añadir a `MH_ARG_MAP`:

```

1 MH_ARG_MAP = {
2     # ...
3     # Lista los strings de params que tu func necesita:
4     'NAE': ('maxIter', 'iter', 'dim', 'population', '
best'),
5     # ...
6 }
7

```

¡Asegúrate de que los nombres en la tupla coincidan **exactamente** con los parámetros que tu función `iterarNAE` usa!

6.3. Paso 3: Habilitar en experimentos

Para usar la nueva MH:

1. Editar `src/util/json/experiments_config.json`:
2. Añadir a "mhs":

```
1 // ...
2 "mhs": ["SBOA", "PSO", "NAE"], // <--- Agrega aqui
3 // ...
4 }
5
```

6.4. Paso 4: Consideraciones adicionales (opcional)

- **Versiones específicas (BEN vs SCP/USCP):** Si necesitas lógica muy diferente, crea funciones separadas (ej. `iterarNAE_Ben`, `iterarNAE_Scp`) y regístralas con claves distintas ("NAE_BEN", "NAE_SCP"). Tengan en cuenta que idealmente debe solo haber una lógica para todos los problemas.
- **Algoritmos binarios (GA):** Pueden requerir manejo especial si no siguen el flujo estándar continuo + binarización.
- **Pruebas:** Verifica tu nueva MH ejecutándola en problemas de prueba.

Siguiendo estos pasos, podrás integrar nuevas metaheurísticas de manera organizada.

7. Análisis de resultados

Tras ejecutar los experimentos y almacenar los resultados en la base de datos SQLite, el siguiente paso es procesar esta información para obtener gráficos y estadísticas comparativas. El proyecto proporciona un sistema modular y automatizado de análisis post-ejecución.

7.1. Estructura del script de análisis

El análisis de resultados está completamente centralizado a través del script:

```
scripts/analisis.py
```

Este script actúa como punto de entrada de la biblioteca modular del análisis (`scripts/analisis_pkg/`), la cual está compuesta por:

- `cache.py`: Gestión de la memoria caché al consultar datos de SQLite para acelerar el procesamiento.
- `config.py`: Carga las configuraciones de directorios, experimentos y el propio análisis.
- `parser.py`: Filtra y valida los nombres de archivos CSV y datos de rendimiento.
- `plotting.py`: Lógica gráfica para boxplots, violinplots y curvas de convergencia.
- `processing.py`: Motor principal que organiza el procesamiento en lotes paralelos usando Joblib para evitar el desbordamiento de memoria RAM.

El análisis consulta los experimentos finalizados directamente desde la base de datos de SQLite, procesando las iteraciones (contenidas en formato CSV como BLOBs) de forma asíncrona y robusta.

7.2. Configuración del análisis

El proceso de análisis se configura mediante archivos JSON ubicados en `src/util/json/`:

- `dir.json`: Especifica las rutas relativas donde se guardan los resultados finales, incluyendo resúmenes de rendimiento (`resumen`), gráficos de convergencia (`"graficos"`), boxplots (`"boxplot"`), violinplots (`"violinplot"`), entre otros.
- `analysis.json`: Este archivo actúa como panel de control para el rendimiento del procesamiento y la visualización gráfica. Su estructura es la siguiente:

```
{
  "performance": {
    "batch_size": 200,          // Tamaño del lote a procesar
    "gc_interval": 500,        // Frecuencia del Garbage Co.
    "ram_warning": 85,          // % de uso de RAM para advertir
    "cache_max_size": 256      // Tamaño máximo de caché en MB
  },
  "graficos": {
    "graficos_por_corrida": false, // Evita generar miles de archivos
    "modo_logaritmico": "auto",    // Escala ('log_scale' o 'linear')
    "dpi": 300                     // Resolución de gráficos
  }
}
```

```
}
```

- `experiments_config.json`: El script de análisis consulta este archivo para obtener la lista de metaheurísticas activas ("`mhs`") e instancias seleccionadas a incluir en las tablas estadísticas comparativas.

7.3. Ejecución del análisis

Para procesar y analizar los resultados obtenidos de la base de datos local:

1. Asegúrese de haber completado y finalizado ejecuciones de experimentos.
2. Desde la terminal en el directorio raíz del proyecto, ejecute:

```
python scripts/analysis.py
```

El script procesará los datos pendientes de la base de datos en lotes, optimizando la memoria RAM, e informando el progreso en tiempo real.

7.4. Salidas generadas

Los resultados procesados se escriben en el directorio `outputs/results/`, organizados en:

- **Tablas estadísticas (`resumen/`):**
 - Archivos CSV con métricas detalladas por metaheurística (mejor fitness, promedio de fitness, desviación estándar, tiempo promedio de ejecución).
 - Estadísticas de exploración y explotación promedio (`%XPL` y `%XPT`) a lo largo de las iteraciones.
- **Visualizaciones gráficas:**
 - **`graficos/`**: Historial de búsqueda de mejores soluciones (fitness vs iteraciones), y gráficos representativos de balance `%XPL` y `%XPT` por algoritmo.
 - **`boxplot/` y `violinplot/`**: Distribución de calidad final de las corridas para cada algoritmo, permitiendo comparar visualmente la consistencia y robustez de los optimizadores.